DATE: PAGE:

SQL

SQL (Structured Query Language) is a standard programming language specifically designed for managing and manipulating relational database. It allows users to create, read, update, and delete data stored in tables

Data

Data refers to raw unprocessed facts and figures that have no context or meaning on their own In databases, data can be numbers, text, dates, images, or any other type of information stored systematicaly. when data is processed and given context, it becomes meaningful information
Ex - Sahana, Maanya, 19, 15. etc.

Database

A database is an organized collection of structured data stored electronically, designed for efficient retrieval, insertion and management It acts as a central repository that allows multiple users and application to access shared data in a controlled and consistant manner

Scanned with OKEN Scanner

DATE: PAGE:

Syntax

1. CREATE - used to create a new database
   CREATE DATABASE database_name;
2. ACCESS - used to select / access the database
   USE database_name;
3. VIEW: used to display all available database
   SHOW DATABASES;
4. DELETE; used to permanently delete a database.
   DROP DATABASE database_name;

DBMS

A database Management System (DBMS) is Software that provides an interface between users / applications and the database. It handles data storage, retrieval, security, and integrity. It eliminates the complexity of managing raw data files, offering tools to define, manipulate, and control data efficiently

Ex: MySQL - open source RDBMS
Oracle - Enterprise-grade DBMS
SQL Server - Microsoft's DBMS

Scanned with OKEN Scanner

DATE: PAGE:

Types of DBMS

- HDBMS (Hierarchical): Data stored in tree structure. Parent-child relationships.
  Ex - IBM, IMS.

- NDBMS (Network): Data stored in graph / network
  Ex : IDMS

- ODBMS (Object-Oriented): stores data as objects
  Ex : db4O, object DB

- RDBMS (Relational): Data stored in tables with rows and columns. It is most widely used world wide languges.
  Ex - MySQL, Oracle.

objectives of SQL

* Mass Storage
* Data Redundancy
* Data Protection
* Platform Independence
* Improved Data sharings

Scanned with OKEN Scanner

DATE: PAGE:

History of SQL

- 1970 - Dr. E.F. Codd proposed the Relational Model at IBM
- 1974 - SQL was developed by Donald. D. Chamberlin and Raymond F Boyce at IBM
- 1979 - Oracle became the first commercial SQL product
- 1986 - SQL became an ANSI standard.

Scanned with OKEN Scanner

DATE: PAGE:

1. what is ER Diagram

ER - Entity - Relationship, An ER diagram is a visual blueprint of a database. we use it before creating the actual database to understand wt data we need and how different data are connected

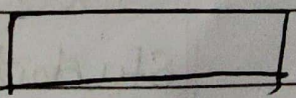
Think of it like: Real world system → ER diagram → database

Ex: for a college database, we may have Students, faculty, course, department, Result.

Componants of ER model

i Entity

An entity is a real world object that we want to store information about

It is represente by a rectangle 

Ex: Student is an entity, faculty, course, department, result

* Entity: who or what are we storing data about

Scanned with OKEN Scanner

DATE: PAGE:

iii Attribute

An attribute describes the properties or characteristics of an entity

It is represented by an oval (⬭)

Ex: The entity student can have

Student = Roll no, name, branch, semester, mobile no, address, age

so, Entity = students

Attribute = student details

Types of attribute

a Single valued attribute

An attribute that has only one value for an entity

Ex: Student Roll no → 101

Student has only 1 roll number

Represented as oval (⬭)

b Multi valued Attribute

An attribute that can have multiple value for same entity

Ex: Student → Phone no.

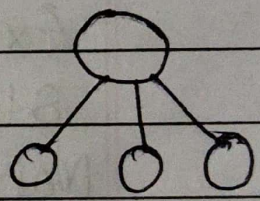
One student can have multiple phone no

Scanned with OKEN Scanner

represented by double oval ( )

c Derived attribute
an attribute whose value is calculated from another attribute
Ex: Date of birth → Age
* if DOB is stored, the system can calculate age
* Age is calculated, not directly stored
* represented by Dashed oval ( )

d Composite Attribute
An attribute that can be divided into smaller sub parts
Ex: Name → first name
→ Middle name
→ last name



Name is made up of multiple smaller attribute Represented as

e Stored attribute
An attribute whose value is directly stored in the database
Ex: DOB = 10/03/2005
DOB is entered and stored directly in the DB represented as oval ( )

Scanned with OKEN Scanner

DATE: PAGE:

iii Relationship

A relationship shows the association b/w 2 or more entities

* It is represented by a diamond (◇) and placed b/w entities, with lines connecting them

* Student – Issues – Book

Here: Student = Entity
Book = Entity
Issuess = Relationship

A student issues a book

Ex: In a college
Student = entity
Name, Roll no, branch = attribute
course = entity
Enrolls = Relationship

Data Types

SQL data types define the kind of values a column can hold, choosing the right data type ensures data integrity, storage efficiency and query performance

DATE: PAGE:

Types of Data types

1 Numeric Data types

- INT / INTEGER : whole numbers
  Eg : 1, 100, -50
- FLOAT / DOUBLE : Decimal numbers
  Eg : 3.14, 99.99
- DECIMAL (p,s) : used when you need accurate decimal values, especially for money
  Ex: DECIMAL (10,2) for currency
- → BIGINT : used for very large whole no.
  Ex : population BIGINT
  population of a large country, very large transaction ID's, etc

2 String Data type

- CHAR (n) : fixed length string, always uses n bytes
  Ex : CHAR (10)
- VARCHAR (n) : variable length string uses actual length
  Ex : VARCHAR (255)

imp point is that it uses storage based on the actual length of the value, rather than always using the full declared length.

Scanned with OKEN Scanner

DATE: PAGE:

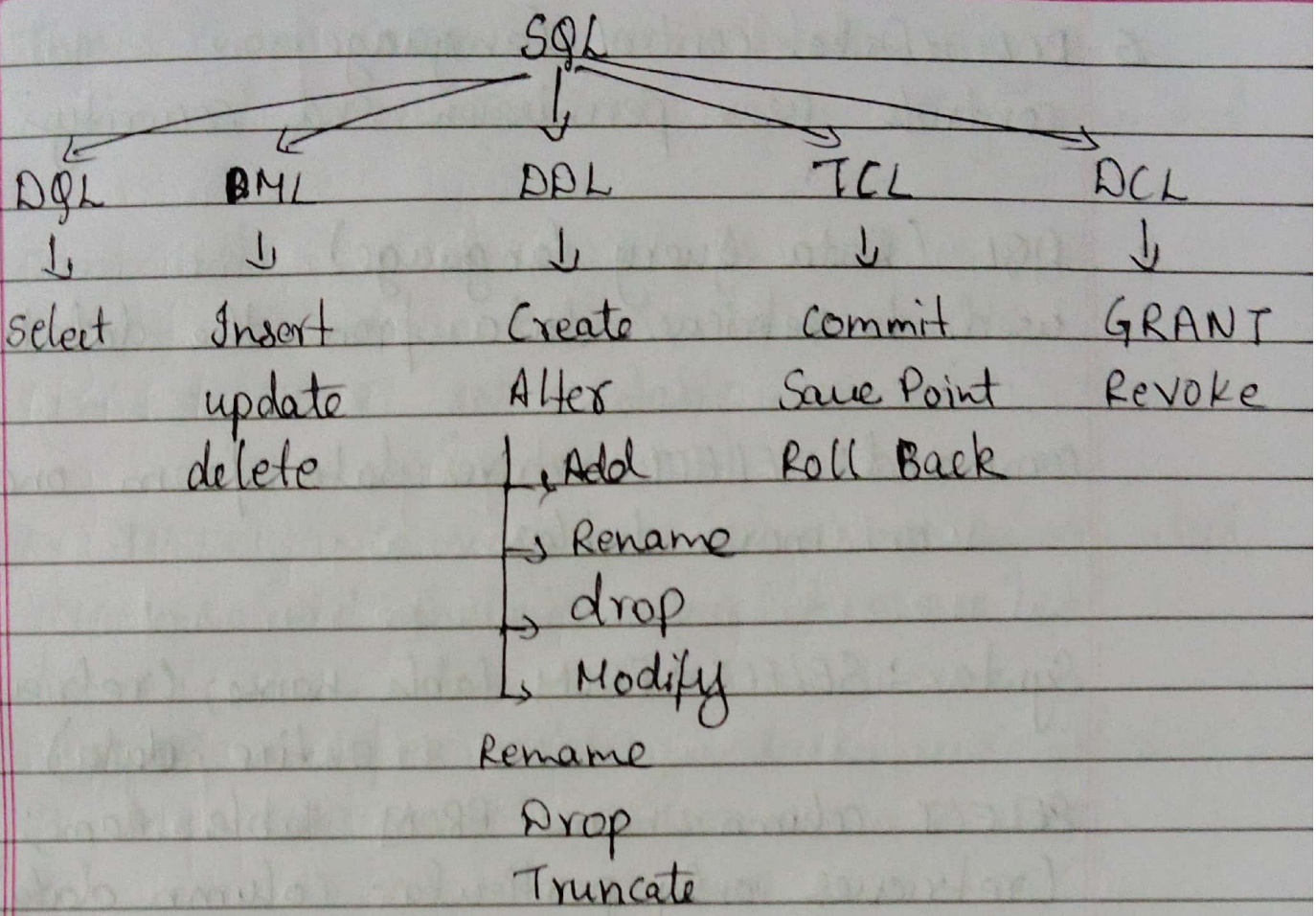
- TEXT : Large text content (upto 65,535 CHARS)
  Eg : Addres
- ENUM : allows to select one value from a predefined list
  Ex: gender ENUM ('Male', 'Female', 'others')

3 Date / Time Data types
- DATE : Stores date only (F): (YYYY-MM-DD)
  Ex : Joining-date DATE
- TIME : Stores time only (F): (HH:MM:SS)
  Ex : 09:30:00
- DATETIME : Stores both date and time
  Ex : 2026-09-22 09:05:25
- YEAR : Stores 4-digit year
  Ex : 2026
- MONTH : Stores only month values upto 2 digit
  Ex : 01, 09, 12

Sub Language of SQL
SQL is divided into 5 sub language, each serving a distinct purpose in database operations. understanding these categories is fundamental to mastering SQL

Scanned with OKEN Scanner

DATE: PAGE:



1 DQL - Data Query Language
Retrieves data from the database

2. DML - Data manipulation Language
Manipulates data stored in tables

3 DDL - Data definition Languge
Defines and modifies the database structure

4 TCL - Transaction control Language
manages transactions to ensure data consistency

Scanned with OKEN Scanner

DATE: PAGE:

5 DCL - Data control language
controls user permission and security

1 DQL (Data Query language)
used to retrieve data from the database

command: SELECT - retrives data from one or more tables

Syntax: SELECT * FROM table_name; (retrieves entire data)
SELECT column_name FROM table_name; (retrieves only particular column data)

Ex: SELECT * FROM employee;
SELECT name FROM employee; (retrives only name column)
SELECT name, salary FROM employee; (retrives multiple columns from table)

2 DML (Data manipulation Language)
This is used to manipulate (change the data stored inside tables
also called as a row level language

Scanned with OKEN Scanner

DATE: PAGE:

* These changes can be rolledback using TCL command like ROLLBACK before being committed

commands

a Insert : adds new row/record to a table

Syntax : INSERT INTO table_name (c1, c2, c3)
VALUES (v1, v2, v3)

Ex: INSERT INTO employee (id, name, salary, dept)
VALUES (4, 'Sahana', 25,000, 'Finance');

b UPDATE : modifies existing data in a table

Syntax : UPDATE table_name
SET column1 = value1,
column2 = value2
WHERE condition;

Suppose Sahana's salary changes from 30,000 to 35,000

Ex: UPDATE employee
SET salary = 35,000
WHERE name = 'Sahana';

IMP

UPDATE employee
SET salary = 35,000

can update every employee salary. so use WHERE

DATE: PAGE:

c DELETE : delete removes rows from a table based on a condition

Syntax: DELETE FROM table_name
WHERE condition;

Ex: DELETE FROM employee
WHERE name = 'Sahana';
Sahana's entire row will be removed

* Be careful with DELETE FROM employee;
This removes all row from the table;

3 DDL (Data definition Language)
used to define and modify the structure of the database objects
DML changes the data
DDL changes the structure
DDL is column level language

a CREATE: is used to create a database object such as a tabel
Syntax: CREATE TABEL tabel_name (
column datatype CONSTRAINTS,
column 2 datatype,

Scanned with OKEN Scanner

DATE: PAGE:

column 3 datatype,
);

Ex: CREATE TABEL employed
id INT,
name VARCHAR (50),
Salary INT
);

This creates a tabel called employee

employee

id	name	Salary

There are no records yet

b ALTER: This modifies an existing database object/ tabel structure

There are several operations under ALERT

i ALERT + ADD: adds a new column

Syntax: ALERT ~~Tal~~ TABLE table-name
ADD column_name datatype;

Ex: ALERT TABEL employee
ADD email VARCHAR (100);

Scanned with OKEN Scanner

DATE: PAGE:

id / name / salary / email
This adds a new column

ii ALTER + RENAME : Changes a column name
Syntax : ALTER TABEL table_name
RENAME COLUMN old_column_name
To new_column_name

Ex: ALTER TABLE employee
RENAME COLUMN name
TO employee_name;

iii ALTER + MODIFY : It modifies either the data type of the existing column when there is no data present in the specific column or helps in changing the position of the column in the ~~tab~~ table

Syntax : ALTER TABLE table_name
MODIFY column_name new_datatype;

Ex: ALTER TABEL employee
MODIFY Salary DECIMAL (10,2);
changes the data type of salary

DATE: PAGE:

Syntax: ALTER TABLE table_name
MODIFY column_name old_datatype
FIRST;

or ALTER TABEL tabel_name
MODIFY column_name old datatype.
AFTER column_name

Ex: ALTER TABLE employee
MODIFY Salary INT FIRST;

Before: id | name | salary | department
After : salary | id | name | department

Ex: ALERT TABLE employee
MODIFY Salary INT AFTER name;

before : salary | id | name | department
after : id | name | salary | department

Ex: ALTER TABEL employee
MODIFY department VARCHAR (50) AFTER name;

id | name | department | Salary

Scanned with OKEN Scanner

DATE PAGE

iv ALTER + DROP : Removes a column completely
Syntax : ALTER table table_name
DROP COLUMN column_name;

Ex : ALTER TABLE employee
DROP COLUMN email;
The email column is no longer exists

c Rename : changes the name of a database object such as table
Syntax : RENAME TABEL old_tabel_name
To new_table_name;

Ex : RENAME TABEL employee
To employees

d Drop : removes the table / object itself, including its structure
deleting the label / database object completely

Syntax : DROP TABLE tabel_name;
Ex : DROP TABLE employee;
Now the tabel itself gone

Scanned with OKEN Scanner

DATE: PAGE:

before : employee
id | name | Salary

after : employee X

e TRUNCATE : removes all rows from the tabel but keeps the tabel structure

Syntax : TRUNCATE TABLE tabel_name;
Ex : TRUNCATE TABLE employee;

before : id	name	Salary
1	Gana	25,000
2	Maan	30,000

after : id | name | salary

DELETE :→ Selected rows or all rows is removed
→ tabel structure remains
→ WHERE condition is used

TRUNCATE : → All rows is removed
→ tabel structure remains
→ NO WHERE condition

Scanned with OKEN Scanner

DATE: PAGE:

DROP: → Entire tabel is removed
→ table structure is removed
→ NO WHERE condition

4 TCL (Transaction control language)
TCL is used to control transactions. ~~manage~~ manage transaction in a database and helps maintain data consistency

A transaction is a group of database operations treated as a unit

Ex: UPDATE employee
SET Salary = 35000
WHERE id = 1;

This is a change ~~mo~~ made during a transaction TCL allows us to decide whether to save the change or undo it

Types
1 COMMIT
2 ROLLBACK
3 SAVEPOINT

Scanned with OKEN Scanner

DATE: PAGE:

1 WHERE clause

The WHERE clause is used to filter records from a table
It tells SQL Give me only the row that satisfy this condition
without WHERE, all rows are returned or affected.

Syntax: SELECT * FROM table_name
WHERE condition;

Ex: SELECT * FROM employees
WHERE dept = sales;
This filters only sales rows

2 OPERATORS
Operators are symbols or keywords used to perform operations on values in SQL

There are 5 Types

1 Arithmetic operators
-> addition (+)
-> Subtraction (-)
-> Multiplication (*)

DATE: PAGE:

-> Division (/)
-> Modulus (%) the returns reminder

Ex: SELECT Salary + 5000
FROM employees;
if Salary = 25000
Result = 30000

2 Relative / comparison operators used to compare 2 values
-> (=) equal to
-> (! = or <>) Not equal to
-> (<) Less than
-> (>) Greater than
-> (<=) Less than or equal to
-> (>=) Greater than or equal to

Ex: SELECT * FROM employees
WHERE Salary > 25000;
This returns employees whose salary is greater than ₹ 25,000

Scanned with OKEN Scanner

DATE: PAGE:

3 Logical operator
used to combine multiple conditions

→ AND : Both condition must be TRUE

```
SELECT * FROM employees
WHERE dept = 'Sales' AND salary > 20000;
```

department must be sales AND salary must be greater than 20,000

→ OR : At least one condition must be TRUE

```
SELECT * FROM employees
WHERE dept = 'Sales' OR dept = 'IT';
```

Department can be sales or IT

→ NOT : means opposit / exclude the condition

```
SELECT * FROM employees
WHERE NOT dept = 'Sales';
```

Give employees who are not in Sales

Scanned with OKEN Scanner

DATE: PAGE:

4 Set operators
used to combine results from queries

→ UNION: combines results and removes duplicate

```
SELECT name FROM employees
WHERE dept = 'Sales'

UNION
SELECT name FROM employees
WHERE salary > 25000;
```

→ UNION ALL: combines results and keeps duplicates

```
SELECT name FROM employees
WHERE dept = 'Sales'
UNION ALL
SELECT name FROM employees
WHERE salary > 25000;
```

5 Special operators

- IN: used when we want to check multiple possible values

DATE: PAGE:

```
SELECT * FROM employee
WHERE dept IN ('Sales', 'IT');
```

→ BETWEEN : used to check whether a value falls within a specific range

```
SELECT * FROM employees
WHERE Salary BETWEEN 20000 AND 30000;
```

→ LIKE : used for pattern matching

```
SELECT * FROM employees
WHERE name LIKE 'S%';
```

% means any no. of characters (S%)
So this can find names starting with S

(%S) → searchs names ends with S
(%S%) → searchs names which contains S anywhere

→ IS NULL : used to check for null values

```
SELECT * FROM employees
WHERE Salary IS NULL;
```

This returns emp whose salary is missing

Scanned with OKEN Scanner

DATE: PAGE:

→ AS : used to give a temporary / a alias name to a column or table

```
SELECT Salary AS monthly-salary
FROM employees;
```

Here monthly-salary is temporary name displayed for salary

3 Literals

Literals is a fixed value written directly in a SQL query. It doesn't change during query execution

There are 3 types of Literals

→ String Literals

Text values are generally written inside quotes

```
SELECT * FROM employees
WHERE dept = 'Sales';
```

'Sales' → string literals

DATE: PAGE:

- Numerical Literal
No. written directly in a query
SELECT * FROM employees
WHERE Salary > 50000;

50000 → Number Literals

→ Date Literals
A date value written directly
SELECT * FROM employees
WHERE doj = '2026-08-21';

'2026-08-21' → Date Literal

Literal	Vs	Variable
* fixed values		* can change
* written directly		* stored temporarily

4 Dual Table
Dual is a special dummy table used when we want to execute a SELECT without retrieving data from an actual table.
It is a ~~dumpy~~ dummy table created for practice it contains only 1 row & 1 column

DATE: PAGE:

Syntax : SELECT data FROM DUAL;
SELECT data;

Ex : SELECT 10+20 FROM DUAL;
o/p : 30
Ex : SELECT 'HELLO' FROM DUAL;
o/p : HELLO

NOTE : operators control logic,
Literals provides values,
Dual helps us practice

5 TRIM
This Removes Leading and or trailing spaces from a string (or specified char) from a string

(LTRIM) ⇒ Removes from the left
(RTRIM) ⇒ removes from right
(TRIM) ⇒ removes from both sides

TRIM Ex : SELECT TRIM (' HELLO ') AS trimmed;
o/p : HELLO

Scanned with OKEN Scanner

DATE: PAGE:

LTRIM Ex: SELECT LTRIM ('SQL') AS left_trimmed;
o/p : SQL

RTRIM Ex: SELECT RTRIM ('SQL ') AS right_trimmed;
o/p : SQL

6 Distinct

This removes duplicate values / rows from the results and returns unique values

Syntax: SELECT DISTINCT column_name
FROM table_name;

Ex:

dept
Sales
IT
Sales
HR
IT

SELECT DISTINT dept → FROM employees;

dept
Sales
IT
HR

Scanned with OKEN Scanner

DATE: PAGE:

CONSTRAINTS

Constraints are rules applied to columns in a table. They help make sure that the data entered into the database is
-> correct, valid, reliable, consistent

They prevent invalid data from being inserted into a table

Ex: Suppose we have employee table

ID	Name	Age
001	Sana	22
002	Maan	25

if we say Age must be greater than 18, SQL can prevent someone from entering Age = 10. That rule is constraint

Types of constraints

a unique
ensures that all values in a column are different

Scanned with OKEN Scanner

DATE: PAGE:

Ex: emp email ID must be different

```
CREATE TABLE Employeel
   email VARCHAR (100) UNIQUE
);
```

Now, Sahana@gmail.com
Sahana@gmail.com
The second value will not be allowed bez it is duplicate

- This ensures all values in a column are different unlike primary key
- a table can have multiple unique constraints like

```
CREATE TABEL Employeel
email VARCHAR (100) UNIQUE
 name VARCHAR (20) UNIQUE
```

a And UNIQE columns can contain NULL (one NULL per column

b Not NULL
The column should not be empty (NULL. Every row must provide a value for this column

DATE: PAGE:

Ex: CREATE TABEL Employees (
name VARCHAR (50) NOT NULL
);

c CHECK
allows us to specify a condition that the data must satisfy

Ex: CREATE TABLE Employee (
age INT CHECK (age >= 18)
);

d DEFAULT
DEFAULT automatically provides a value when we don't ~~one~~ give one during insertion INSERT

CREATE TABLE Employee(
name VARCHAR (50),
city VARCHAR (30) DEFAULT 'Banglore'
);

e PRIMARY KEY
uniquely identifies each row in a table
combination of not null and ~~uni~~

DATE: PAGE:

unique. Each table can have only one primary key

```
CREATE TABLE EMPLOYEE (
    employee_id INT PRIMARY KEY,
    name VARCHAR(50)
);
```

F FOREGIN KEY

Creates a link blw 2 tables by referencing the primary key of another table.
It enforces referential integrity ensures that value in a column match values in the referenced table

Ex: Imagin we have 2 tables

Table 1: Employee

employee_id	name
101	Sahana
102	Maanya
103	Sanjana

Here, employee_id is the primary key

Scanned with OKEN Scanner

DATE: PAGE:

Table 2 : Attendance

attendance_id	employee_id	status
1	101	Present
2	102	Absent

Here, employee_id is the foregin key

It refers to: Attendance.employee_id
↓
Employee.employee_id

```
CREATE TABLE Attendance (
    Attendance_id INT PRIMARY KEY,
    employee_id INT,
    Status VARCHAR (20),
    FOREGIN KEY (employee_id)
    REFERENCES Employee (employee_id)
);
```

Scanned with OKEN Scanner